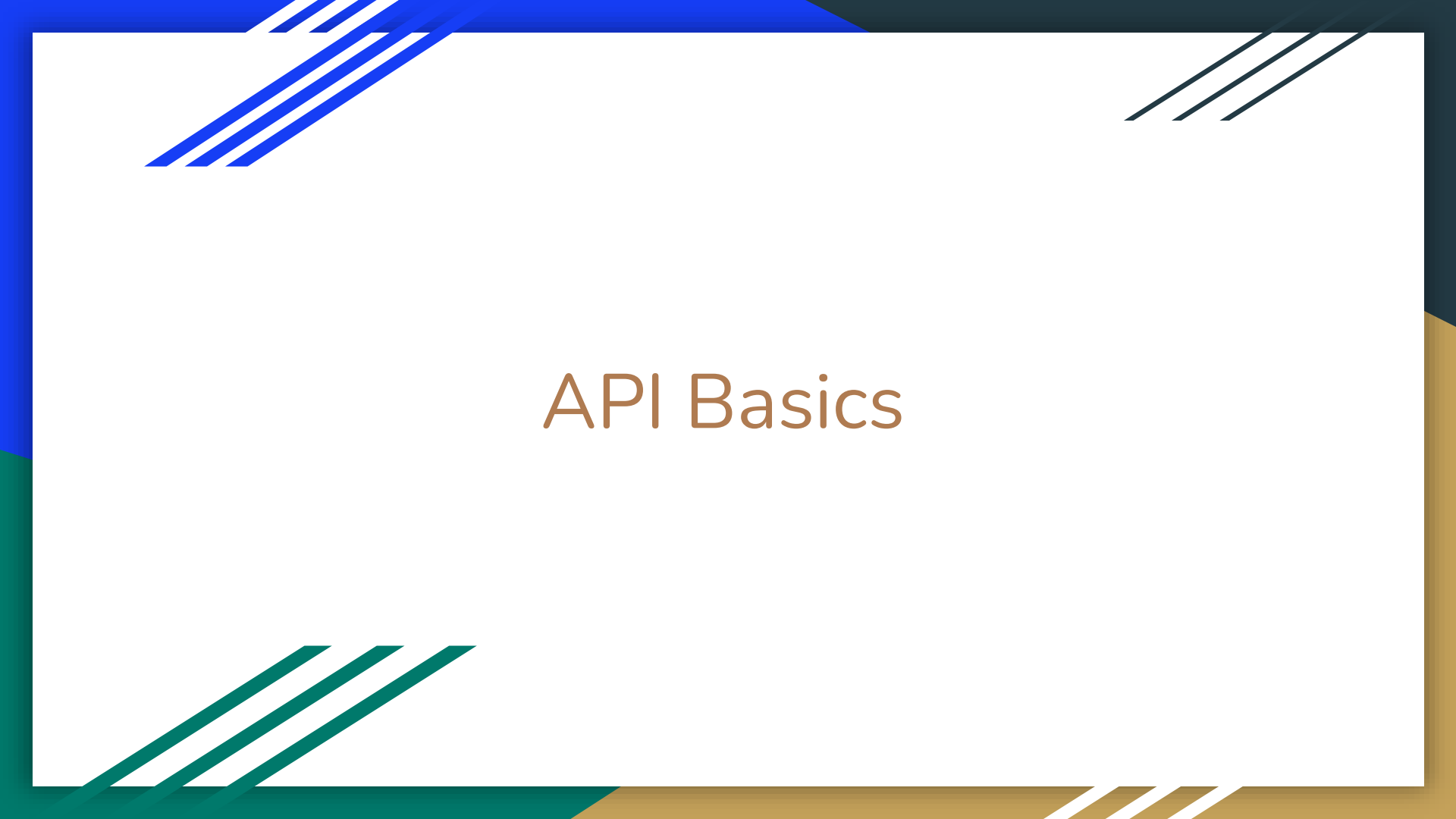




API Basics

An API is like...

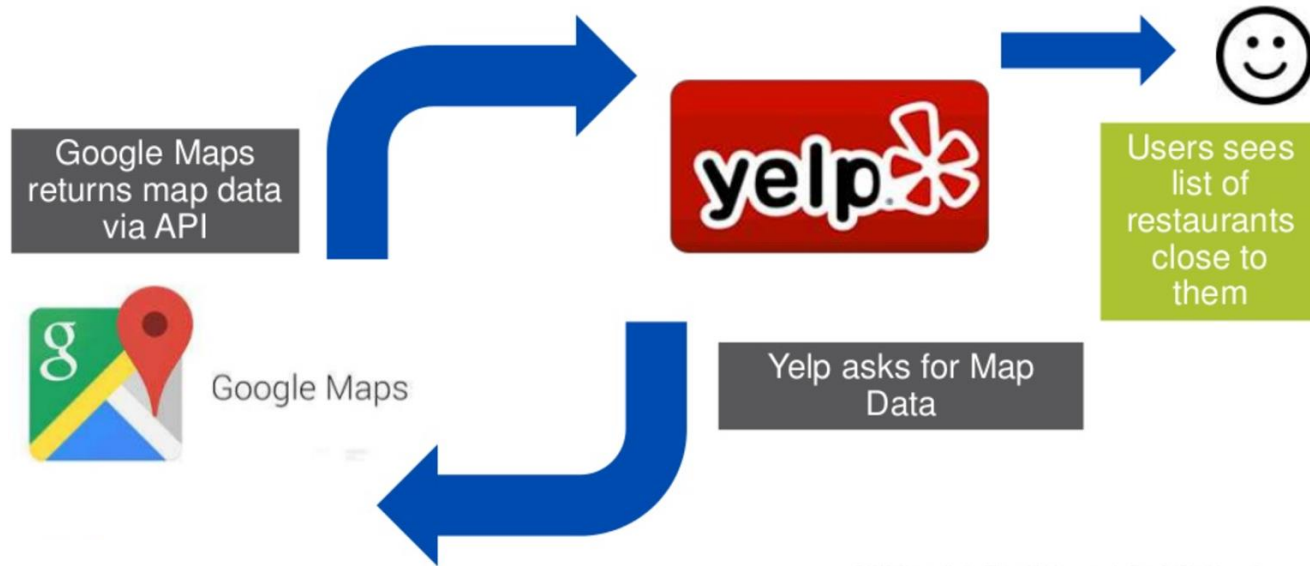


An API (Application Programming Interface) is best thought of as a contract provided by one piece of computer software to another.

The Role of APIs in Modern Software

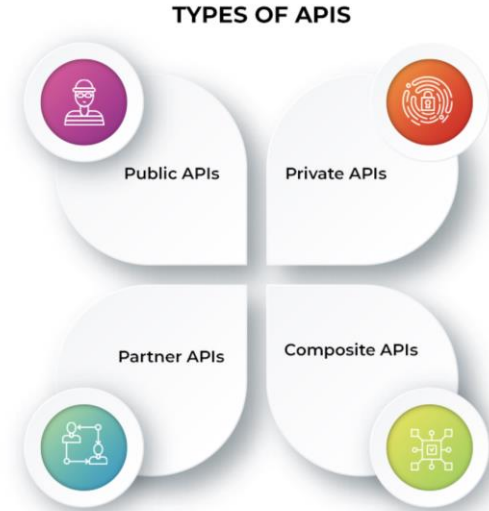
- Application Programming Interfaces (APIs) serve as the communication layer between different software applications, enabling them to exchange data and functionality seamlessly. By abstracting the underlying implementation, APIs allow for interoperability between diverse systems and platforms, fostering innovation and efficiency in software development.

APIs help developers create apps that benefit the end user



Types of APIs

- APIs come in various forms, tailored to different use cases: Public APIs open services to external developers, Private APIs facilitate internal application integration, and Partner APIs enable controlled access between businesses. Each type serves distinct purposes, from expanding brand reach to enhancing internal productivity.



Public API

- Public APIs, also known as External or Open APIs, are exposed to third-party developers and other external users to enable access to a provider's services or data. These APIs are designed with the intent to extend the functionality of external applications, foster innovation, and create new business opportunities. They are meticulously documented and often come with developer support to encourage widespread use. However, managing a public API requires rigorous security measures, throttling, and version control to handle the wide range of applications and potentially large volumes of requests.

Private API

- Private APIs, or Internal APIs, are developed for use within an organization, allowing different teams or systems to communicate and share data securely. These APIs are not exposed to external users, focusing instead on improving productivity, efficiency, and facilitating seamless integration across internal software applications. By using private APIs, businesses can optimize workflows, automate processes, and ensure that sensitive data remains within the secure confines of the organization's IT infrastructure.

Partner API

- Partner APIs are specifically designed for strategic business partners and provide more controlled access than public APIs. They enable direct integration between an organization and selected external parties, facilitating closer collaboration and offering mutual benefits such as shared services, data exchange, and value creation. Access to partner APIs is typically governed by strict contractual agreements, ensuring security and compliance while fostering a trusted partnership ecosystem.

Composite API

- Composite APIs combine multiple API calls into a single call, streamlining client-server interactions, especially in complex operations. This approach reduces the load on the network, improves the speed of data retrieval, and enhances the user experience by minimizing the number of requests and responses between the client and server. Composite APIs are particularly useful in microservices architectures and web applications where multiple services need to be coordinated, providing a more efficient and coherent data processing method.

RESTful APIs vs. SOAP APIs

- RESTful APIs and SOAP (Simple Object Access Protocol) APIs represent two main approaches to web services. RESTful APIs, oriented around web resources and stateless communication, offer simplicity and flexibility. In contrast, SOAP APIs, defined by strict standards and XML-based messaging, provide a high level of security and transactional reliability.



GraphQL: An Alternative to RESTful

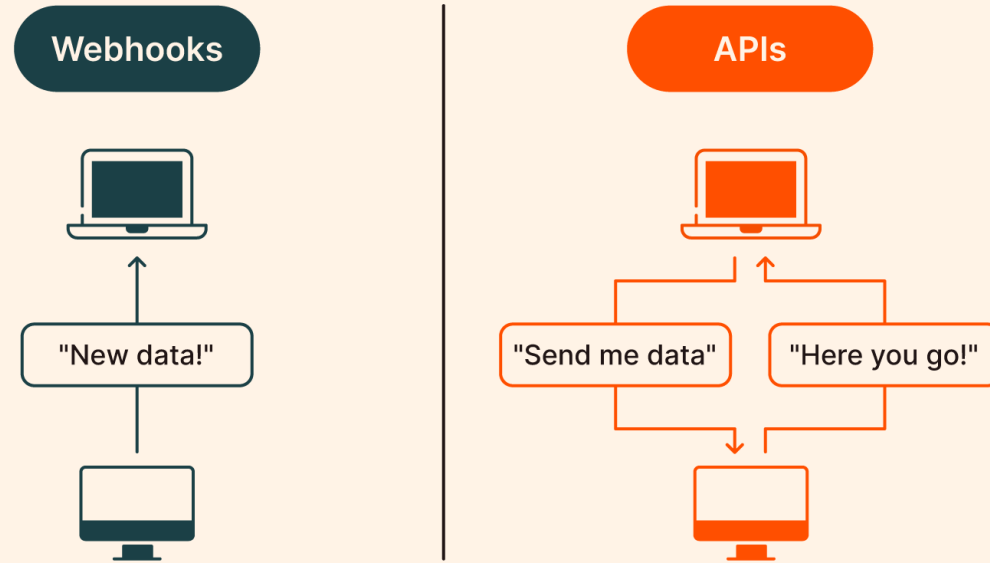
- GraphQL presents an alternative to RESTful APIs by allowing clients to request exactly the data they need, reducing over-fetching and under-fetching. This query language for APIs offers a more efficient, powerful, and flexible approach to data retrieval, catering to the dynamic needs of modern applications.



Understanding Webhooks

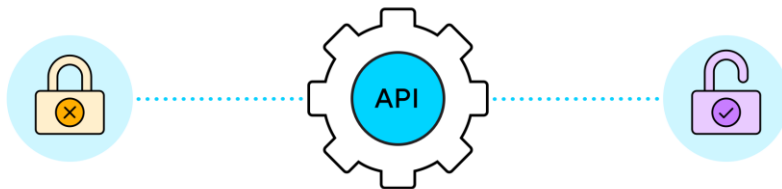
- Webhooks are user-defined HTTP callbacks, providing a mechanism for applications to receive real-time notifications about events in other services. Unlike APIs, which require polling for data, webhooks push data to a specified URL in response to specific triggers, enhancing efficiency and immediacy in application integration.

Webhooks vs APIs



Fundamentals of API Authentication

- API authentication verifies the identity of the caller, ensuring that a request is made by a known entity. Common methods include Basic Authentication, using a username and password; API keys, unique identifiers given to applications; and more sophisticated protocols like OAuth for delegated authorization.



OAuth in Detail

- OAuth is a widely adopted protocol that allows third-party services to exchange web resources on behalf of a user. It provides a secure, efficient way for users to grant applications access to their information without revealing their passwords, using tokens to represent the authorization granted.

Implementing JWT for Secure API Access

- JSON Web Tokens (JWT) offer a compact, URL-safe means of representing claims to be transferred between two parties. In API security, JWTs are used to encode user information, which the server validates to authorize access, making them especially useful in single sign-on (SSO) and token-based authentication.

API Keys: Usage and Security

- API keys act as simple identifiers to control access, commonly used in public APIs for identifying the calling application. While easy to implement, they should be protected from exposure to prevent unauthorized use, often by keeping them out of codebases and using secure storage solutions.

Understanding Access Tokens and Refresh Tokens

- Access tokens are short-lived tokens used to make authenticated API requests, while refresh tokens, with a longer lifespan, are used to obtain new access tokens. This mechanism reduces the risk of token theft and ensures that users can maintain their sessions without repeatedly logging in.

Securing APIs with SSL/TLS

- Encrypting traffic between the client and server using SSL/TLS is a foundational security measure for APIs. It ensures that data exchanged is secure from eavesdropping and tampering, a must-have for transmitting sensitive information like authentication tokens over the internet.



API Security Best Practices

- Beyond authentication and authorization, securing APIs involves a comprehensive approach including validating and sanitizing input data, implementing proper error handling to avoid leaking information, and regularly auditing and testing APIs for vulnerabilities to keep security measures up-to-date and effective.

API Rate Limiting and Throttling

- Rate limiting and throttling are essential for managing the load on an API and preventing abuse. By controlling the number of requests a user can make within a certain timeframe, these practices help maintain the API's reliability and performance, ensuring fair usage and availability for all consumers.

The Anatomy of an API Request

- An API request comprises several key components: the endpoint URL, representing the resource; the HTTP method, indicating the action; headers, containing metadata; and the request body, holding the data to be sent for POST and PUT requests. Understanding these elements is crucial for effective API interaction.



Interpreting API Responses

- API responses return status codes, headers, and body content, providing feedback on the request's outcome. Status codes, like 200 OK or 404 Not Found, indicate success or failure, while the response body, typically in JSON or XML format, carries the requested data or details of the error encountered.

```
# Request
curl -H "Authorization: Bearer <personal_access_token>" "https://app.asana.com/api/1.0/tasks/1224?opt_fields=name,notes"

# Response
HTTP/1.1 200
{
  "data": {
    "name": "Make a list",
    "notes": "Check it twice!",
    "id": 1224
  }
}
```

The Importance of API Documentation

- Comprehensive API documentation is crucial for developer adoption and effective use of APIs. It should detail available endpoints, request/response formats, and authentication requirements, providing real-world examples. Good documentation acts as both a guide and a reference for developers, facilitating a smoother integration process

Versioning Strategies for APIs

- API versioning is vital for managing changes and ensuring backward compatibility. Strategies like URI path versioning, query string parameters, or custom request headers allow developers to introduce new features or changes without disrupting existing integrations, maintaining a contract with API consumers.

The Future of APIs

- The evolution of APIs continues to shape the digital landscape, with trends towards more decentralized approaches, like microservices, and the adoption of AI and machine learning to enhance API functionalities. The future of APIs lies in their ability to adapt, offering more personalized, intelligent, and autonomous capabilities to meet the growing demands of technology advancements.

API Testing and Best Practices

- Effective API testing ensures that APIs perform as expected, covering functionality, reliability, performance, and security. Best practices include comprehensive test planning, automation, environment management, and continuous monitoring. Ensuring quality across the API lifecycle is essential for delivering reliable and robust



REST API

Introduction to RESTful APIs

- RESTful APIs stand as the cornerstone of web communication, enabling different software systems to exchange data over the internet in a seamless manner. By adhering to REST principles, these APIs support the creation of web services that are scalable, reliable, and easy to consume, making them integral to modern web development and application integration.

REST API

A REST API defines a set of functions which developers can perform **requests** and receive **responses** via **HTTP** protocol such as **GET** and **POST**.

REST is best explained as a way to talk to other machines through the server with an HTTP Protocol

Using Verbs (actions) and Nouns (items)

Understanding REST

- Representational State Transfer (REST) is an architectural style designed for distributed systems, particularly the web. It emphasizes simplicity, scalability, and performance. RESTful systems operate over HTTP requests to access and use data. The principles of REST, including statelessness, uniform interface, cacheability, and layered system, guide the development of web services that are easy to maintain and scale.

REST API vs. RESTful API: Understanding the Difference

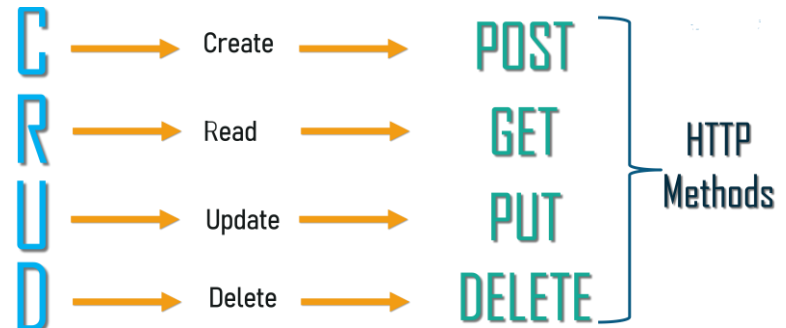
- The terms REST API and RESTful API are often used interchangeably in the industry, but they highlight different aspects of the same concept. REST, standing for Representational State Transfer, is an architectural style defined by six guiding constraints for creating web services. A REST API is any API that adheres to these REST architectural constraints. On the other hand, the term RESTful is more of a qualifier indicating that an API truly embodies and follows the principles of REST. Essentially, while all RESTful APIs are REST APIs, not all REST APIs fully embrace and implement the REST principles to be considered RESTful. This distinction underscores the importance of adherence to REST principles — including statelessness, client-server architecture, cacheability, layered system, code on demand (optional), and uniform interface — in developing truly RESTful services.

Key Characteristics of RESTful APIs

- RESTful APIs are defined by characteristics such as statelessness, where each request from client to server must contain all the information needed to understand and complete the request. They also adhere to a uniform interface, ensuring a standardized way of communicating between client and server. This uniformity allows for decoupling of the architecture, which enhances the scalability and performance of web services.

CRUD operations in RESTful APIs

- CRUD operations form the cornerstone of interaction with RESTful APIs, mirroring the basic actions databases perform on data: Create, Read, Update, and Delete. In the context of RESTful APIs, these operations correspond directly to HTTP methods, ensuring a standardized way of handling resources. Create operations use the POST method to create new resources. Read operations retrieve data with the GET method, making them idempotent and safe, meaning multiple requests have the same effect as one. Update operations modify existing resources and can be achieved through PUT or PATCH methods, with PUT replacing the entire resource and PATCH applying partial updates. Lastly, Delete operations remove resources and are implemented with the DELETE method. This mapping provides a clear, intuitive way to manage resources on the web, allowing developers to perform database-like transactions over the internet with ease and precision.



Resource Identification in REST

- In RESTful APIs, resources are identified using Uniform Resource Identifiers (URIs), typically in the form of URLs. A well-designed REST API provides intuitive and consistent access to resources by using URIs to represent individual entities or collections of entities in a hierarchical structure. This approach ensures that interactions with the API are straightforward and resource-oriented.

Statelessness in RESTful Architecture

- The statelessness of RESTful APIs means that each request from a client to a server must contain all the information the server needs to fulfill the request. The server does not store any state about the client session on the server side. This design principle simplifies the server architecture, improves scalability, and enhances reliability and performance.

RESTful API Response Formats

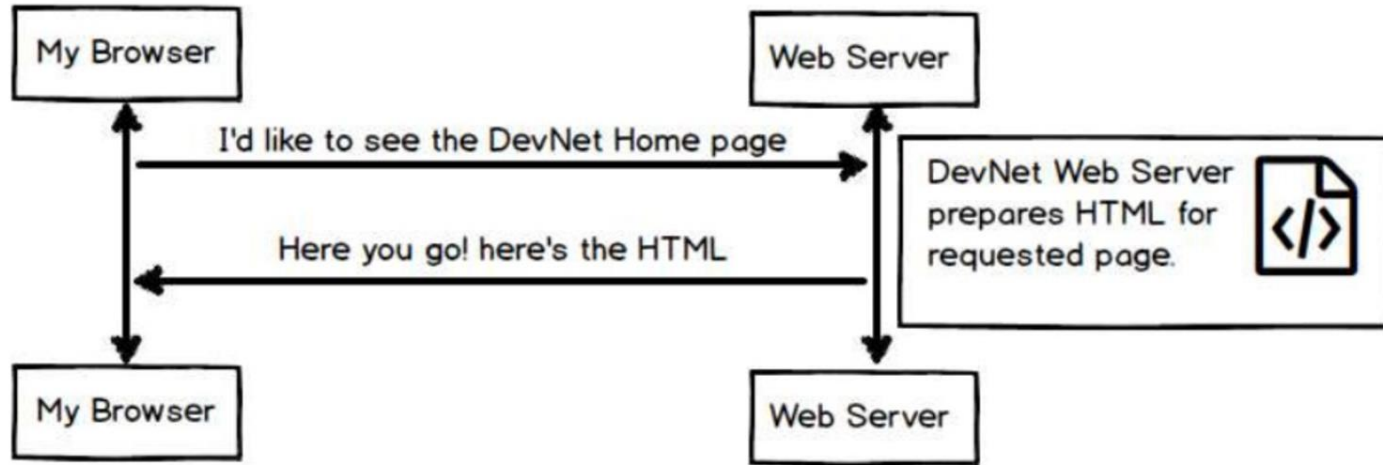
- RESTful APIs commonly use JSON (JavaScript Object Notation) or XML (Extensible Markup Language) as their communication language, allowing for efficient, platform-independent data exchange. JSON, in particular, has gained popularity due to its lightweight nature and ease of use with JavaScript, making it a preferred choice for web applications.

Security Considerations for RESTful APIs

- Securing RESTful APIs is crucial to protect sensitive data and ensure safe communication between client and server. Common security practices include using HTTPS for encrypted communication, implementing authentication tokens, and applying access control mechanisms to restrict resource access. Additionally, input validation and regular security testing are essential to mitigate vulnerabilities.

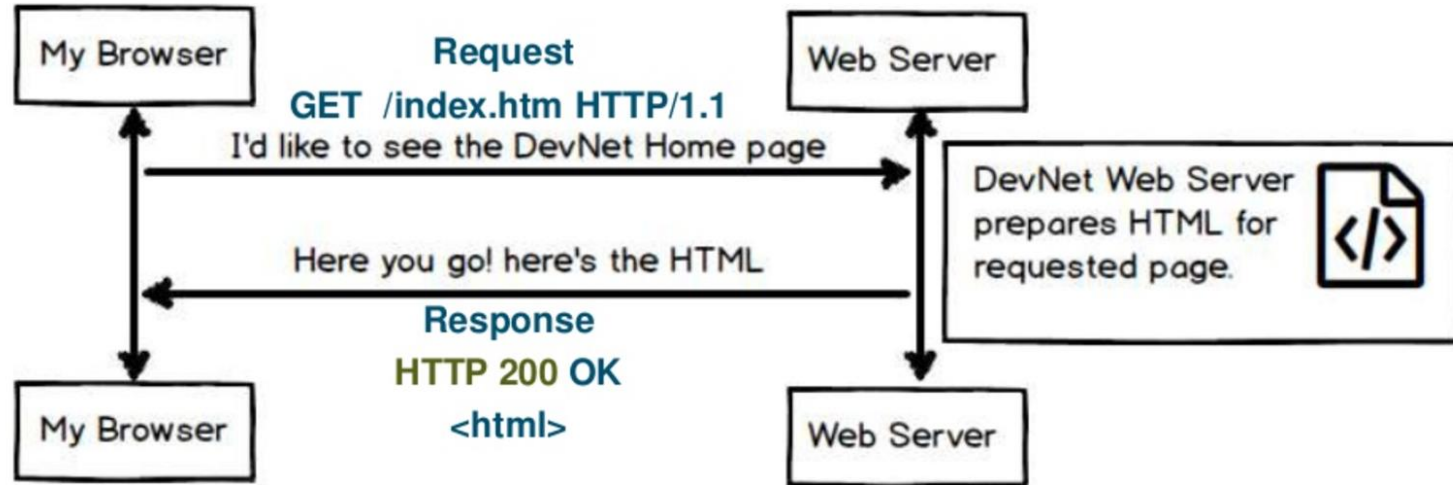
Know What Request and Responses are

View a Web Page



Know What Request and Responses are

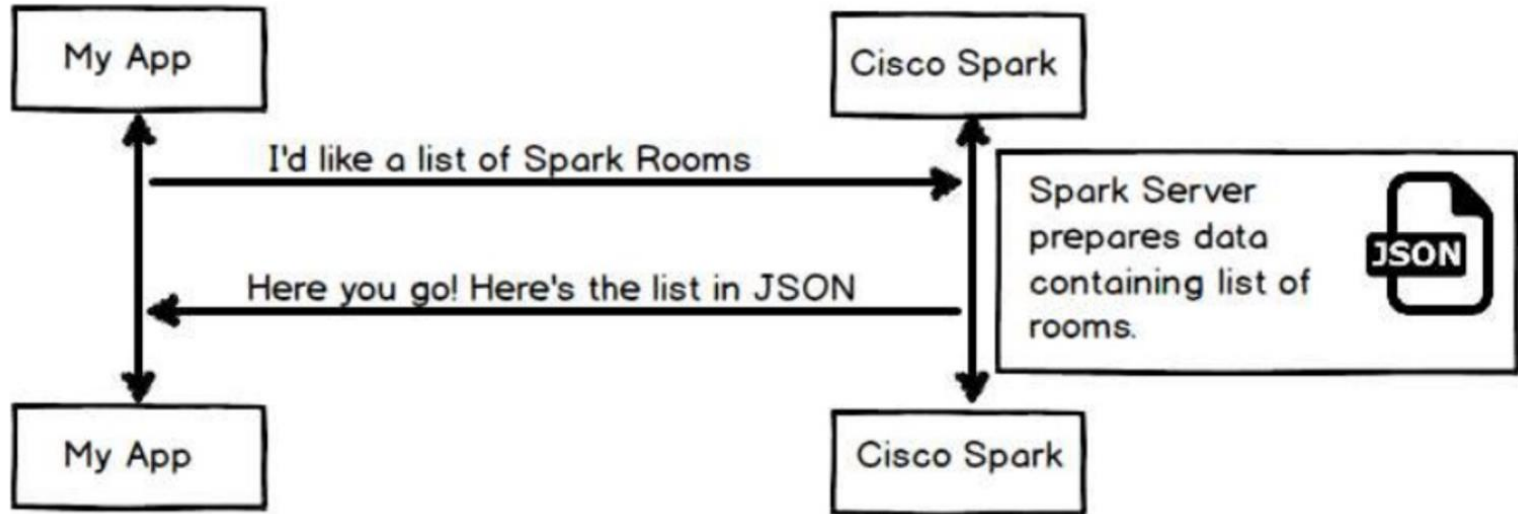
View a Web Page



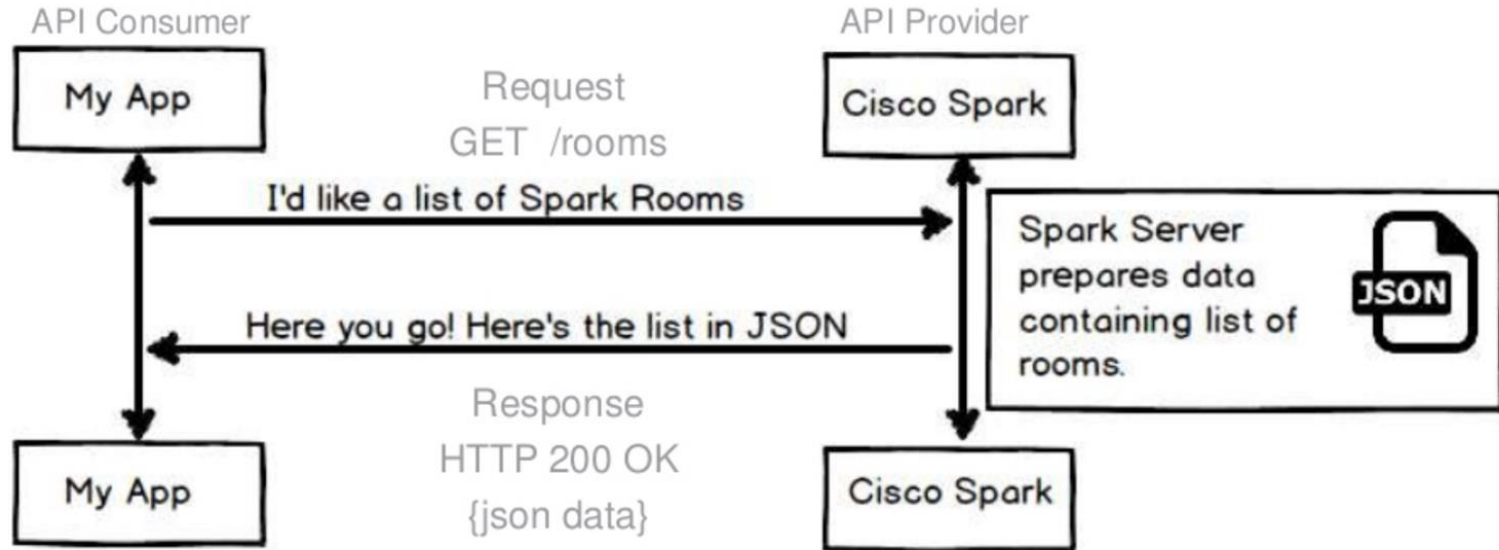
REST APIs use Request and Response too



Get Data using an API (Spark)



Get Data using an API – Response!



List of Verbs!

List items

GET /items

Create an item

POST /items

Get an item

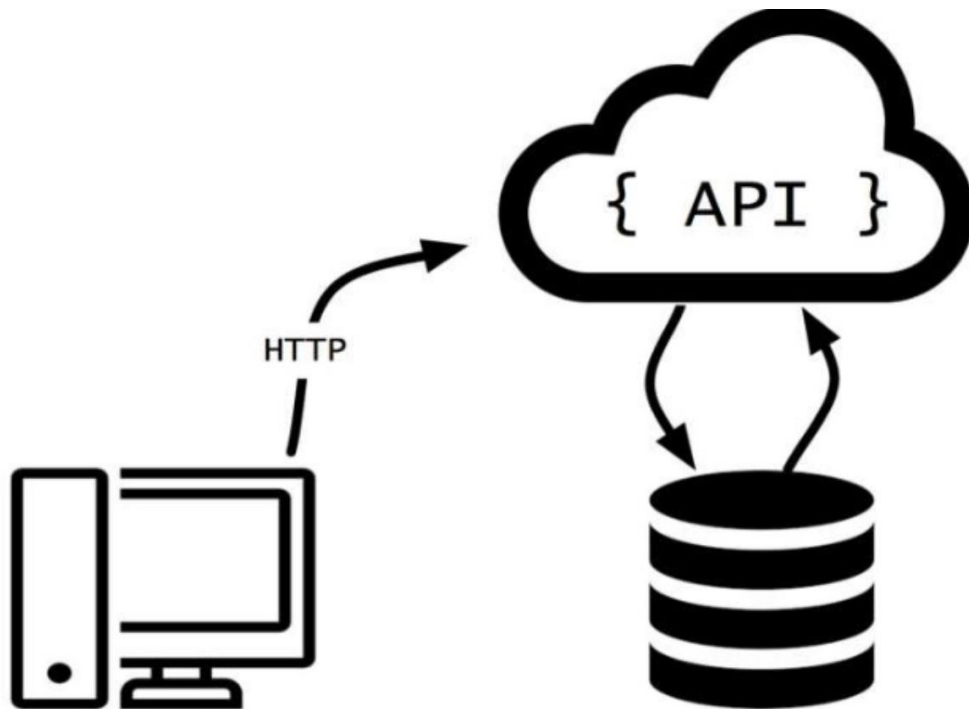
GET /items/{id}

Update an item

PUT /items/{id}

Delete an item

DELETE /items/{id}



Documentation is Your Dictionary

API Reference for Rooms (Verbs and Nouns)

 Spark for Developers

DocumentationBlogSupportHausFun

API REFERENCE

People

Rooms

Memberships

Messages

Teams

Team Memberships

Webhooks

Organizations

Licenses

POST	https://api.ciscospark.com/v1/people	Create a Person
GET	https://api.ciscospark.com/v1/people/{personId}	Get Person Details
PUT	https://api.ciscospark.com/v1/people/{personId}	Update a Person
DELETE	https://api.ciscospark.com/v1/people/{personId}	Delete a Person
GET	https://api.ciscospark.com/v1/people/me	Get My Own Details

Rooms

Rooms are virtual meeting places where people post messages and collaborate to get work done. This API is used to manage rooms themselves. Rooms are created and deleted with this API. You can also update a room to change its title, for example.

Request Parameters for “MESSAGES”

Request Parameters

Name	Type	Description
roomId	string	The room ID.
toPersonId	string	The ID of the recipient when sending a private1:1 message.
toPersonEmail	string	The email address of the recipient when sending a private 1:1 message.
text	string	The message, in plain text. If <code>markdown</code> is specified this parameter may be <i>optionally</i> used to provide alternate text for UI clients that do not support rich text.
markdown	string	The message, in markdown format.

Designing RESTful APIs

- Designing a RESTful API involves careful planning and adherence to REST principles. This process includes defining resource names, determining resource relationships, and selecting appropriate HTTP methods for actions. Effective API design fosters usability, maintainability, and scalability, ensuring that the API meets the needs of both the developers and the end-users.



Tools

Learn How to Use Debug/Test Tools for APIs

Tools to Debug

- Webhook Debugging (RequestBin formerly requestbin.org...)
- Webhook Utilities (Torpio...)
- Local Tunneling (ngrok...)
- API Monitoring (Runscope...)
- Response Mocking (mocky.io...)
- JSON Utilities (JSONFormat...)
- OAUTH Utilities (oauth.io...)
- API Directories (APIS.io, ProgrammableWeb...)
- API Testing (Runscope Radar...)
- Load Testing (loader.io...)
- GUI HTTP Clients (POSTMAN...)

POSTMAN



Spark stsftartz

List DevNet repos (JSON)

method → GET

url → `https://api.github.com/users/CiscoDevNet/repos?page=1&per_page=2`

query parameters → Params

request headers → Headers (1)

response body → Body

response headers → Headers (23)

status code → Status: 200 OK

content-type → JSON

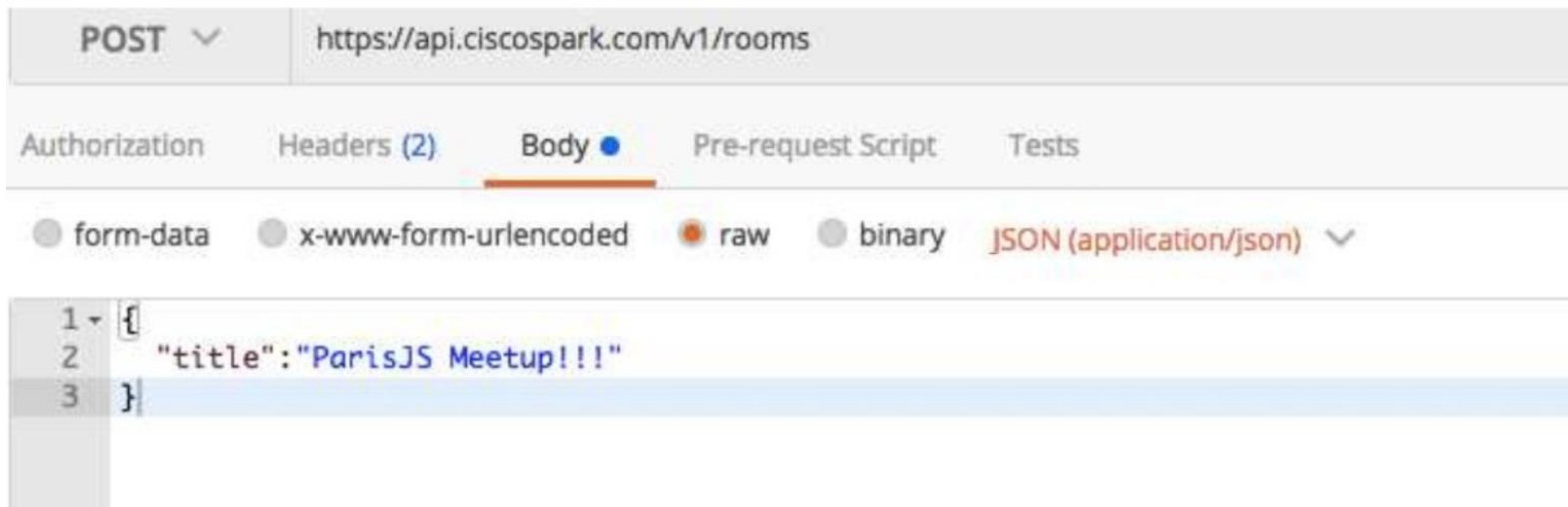
request headers table:

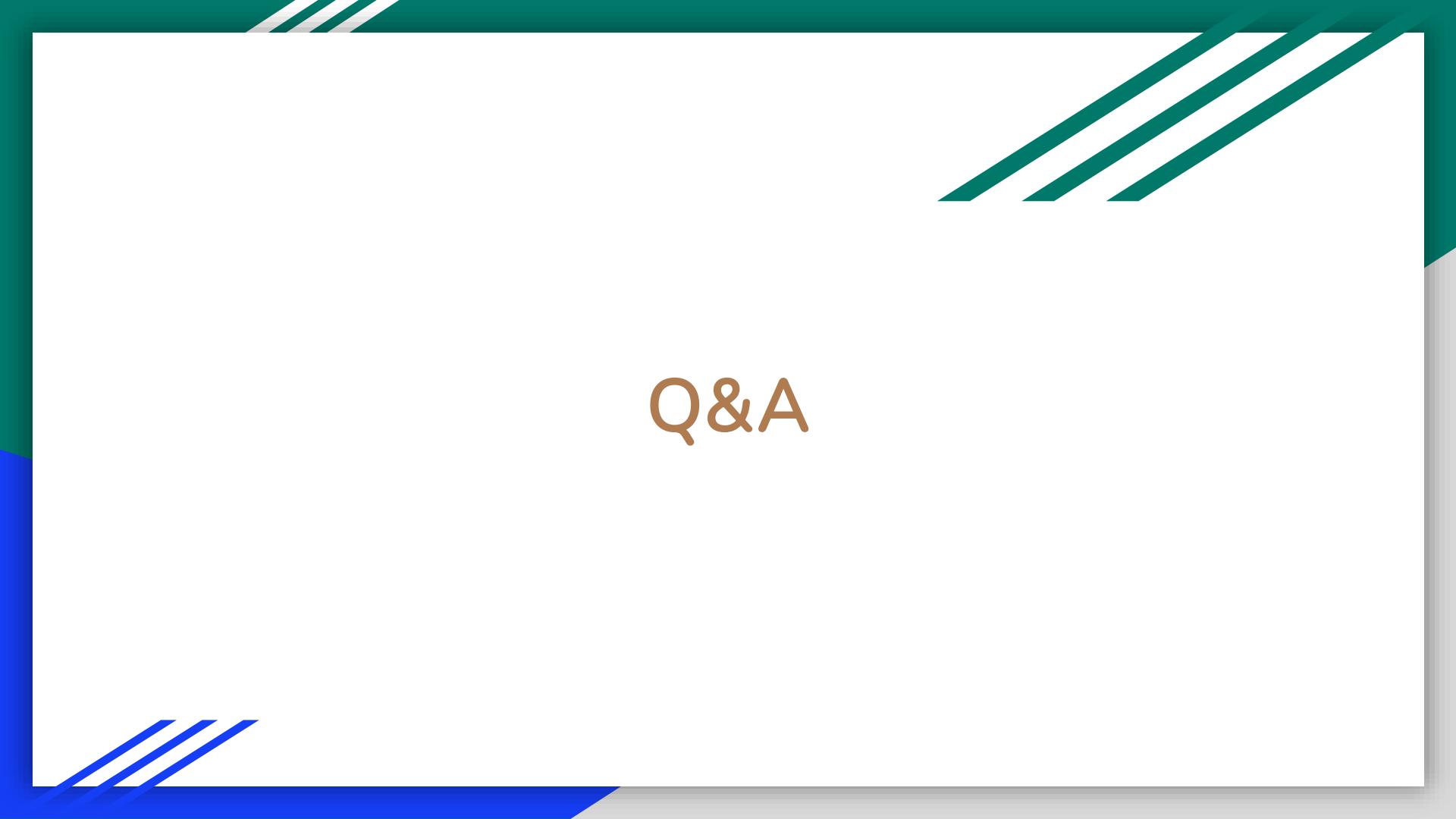
key	value
Accept	application/json

response body (JSON):

```
[{"id": 1, "name": "Repo 1"}, {"id": 2, "name": "Repo 2"}]
```

Add JSON to the Body in POSTMAN





Q&A